SYSTEM DESIGN SCENARIO DRILLS

Read only the scenario and interview question. Speak for 60-90 seconds, then compare the answer. Mark the three boxes after a first correct answer, a next-day recall, and a seven-day recall. Prefer evidence and containment before confident guesses.

1. Production Incidents and Latency

S001 Latency after a deployment [ ] [ ] [ ]

Situation: Your team deployed a new Spring Boot version, and 20 minutes later production latency increased sharply. You suspect the deployment, but you are not completely sure.

"What do you think is happening, and what should we do next?"

Answer: The timing makes a release regression the leading hypothesis, but I would verify correlation before declaring causation. I would compare pre- and post-deployment latency, error rate, CPU, memory, garbage collection, database time, and downstream calls, and split metrics by old and new instance version if both are running. If the new version is clearly worse and users are affected, I would pause the rollout or roll back first. After recovery, I would use traces, logs, configuration differences, and code changes to find the regression and add a canary or automated rollback guard.

S002 Peak-hour response time spike [ ] [ ] [ ]

Situation: A service responds in 200 ms under normal traffic, but response time rises to 3-4 seconds during peak hours.

"How would you investigate this performance issue, and what would you check first?"

Answer: I would first locate where the extra latency is introduced instead of assuming the service only needs more servers. I would compare normal and peak traffic using percentile latency and distributed traces, then check request concurrency, CPU, memory, garbage collection, thread pools, connection pools, database time, cache hit rate, queue depth, and downstream latency. If instances are healthy, I would inspect shared dependencies for slow queries, locks, exhausted connections, retries, or rate limits. The fix follows the bottleneck: scale, optimize queries, tune pools, improve caching, shed load, or change a synchronous dependency.

S003 High p99 but healthy average [ ] [ ] [ ]

Situation: Average latency is 180 ms, but p99 latency is above 5 seconds and users report occasional freezes.

"Why can the average look healthy, and how would you find the tail-latency cause?"

Answer: The average hides a small but important group of very slow requests. I would segment p95 and p99 by endpoint, availability zone, instance, tenant, payload size, cache outcome, and downstream dependency. Traces from slow requests can reveal cold cache misses, large responses, database locks, garbage-collection pauses, connection waits, retries, or one unhealthy shard. I would avoid optimizing the common fast path until I know what distinguishes the tail. The immediate mitigation may be a timeout, concurrency limit, or unhealthy-instance removal; the permanent fix targets the slow cohort and gets its own percentile alert.

S004 CPU is low but latency is high [ ] [ ] [ ]

Situation: Request latency is rising, yet application CPU remains below 35 percent on every instance.

"What could be wrong if CPU is not saturated?"

Answer: Low CPU suggests requests may be waiting rather than computing. I would inspect database and HTTP connection pools, thread-pool queues, lock contention, disk or network I/O, downstream latency, DNS, queue waits, and timeout-driven retries. A blocked thread dump and trace waterfall are more useful here than adding CPU. I would also check whether a concurrency limit is too small or whether a dependency is rate-limiting us. If waiting is confirmed, scaling application instances alone may increase pressure on the shared bottleneck, so I would contain traffic and fix or isolate the constrained dependency.

S005 Error rate rises in only one availability zone [ ] [ ] [ ]

Situation: Overall error rate is moderate, but dashboards show most failures come from one availability zone.

"How would you respond without causing a larger outage?"

Answer: I would treat the zone as a suspected failure domain and compare instance health, network errors, dependency reachability, deployment version, and capacity across zones. If evidence is strong, I would gradually drain traffic from that zone while confirming the remaining zones have enough headroom; an immediate full shift could overload them. I would keep writes and data consistency in mind if stateful components are involved. After stabilization, I would identify whether the cause was infrastructure, configuration drift, or a zonal dependency, then test automatic failover and reserve capacity so future evacuation is safe.

S006 Memory grows until instances restart [ ] [ ] [ ]

Situation: A Java service slowly consumes more memory over several hours and is eventually restarted by the platform.

"How would you distinguish a memory leak from normal caching or load growth?"

Answer: I would compare heap usage after full garbage collections, allocation rate, live-object count, traffic, and cache size over time. A healthy cache should plateau or evict; a leak shows retained memory continuing to grow even after collection and after traffic falls. I would capture heap histograms or dumps near different points and compare dominant object retainers, while checking recent code and configuration changes. To contain impact, I might lower traffic per instance, cap caches, or restart gradually. The fix must remove the retention path and add a long-duration soak test rather than relying on restarts.

S007 Intermittent timeout bursts [ ] [ ] [ ]

Situation: The API is healthy most of the day but experiences two-minute bursts of timeouts every hour.

"What patterns would you investigate?"

Answer: The periodic shape suggests scheduled work or synchronized behavior. I would align timeout timestamps with batch jobs, cache expiry, token refresh, log rotation, backups, autoscaling, certificate checks, garbage collection, and downstream maintenance. Traces and queue or connection-pool metrics should show whether requests wait locally or in a dependency. I would also check for many instances performing the same task at once. If synchronization is the cause, I would add jitter, spread batch work, use stale cache values, or isolate background workloads. I would preserve a timeline so the next recurrence can confirm the hypothesis.

S008 Database latency suddenly doubles [ ] [ ] [ ]

Situation: Application code is unchanged, but database query latency doubles within ten minutes.

"What would you check before scaling the database?"

Answer: I would first identify whether all queries slowed or only a query shape, table, shard, or replica. I would inspect active sessions, locks, connection count, slow-query plans, cache hit ratio, disk latency, replication lag, recent schema or statistics changes, backups, and traffic mix. A changed execution plan or blocking transaction can create the symptom without a capacity shortage. I would cancel or isolate harmful work if safe, route reads away from an unhealthy replica, and protect the database from retry storms. Scaling is appropriate only after evidence shows sustained resource saturation rather than contention or a bad plan.

S009 Service is fast locally but slow in production [ ] [ ] [ ]

Situation: An endpoint takes 80 ms in local testing but more than one second in production.

"How would you explain and investigate the difference?"

Answer: Local tests usually omit production data volume, network hops, contention, security layers, cold caches, and real downstream services. I would reproduce the production request shape and use a distributed trace to separate application time from database, cache, network, and dependency time. I would compare configuration, JVM settings, indexes, payload sizes, and data skew, and avoid using production directly for experiments. A realistic staging load test or replay of sanitized traffic can confirm the bottleneck. The fix should address the production constraint and add a representative performance test so local speed is not mistaken for system speed.

S010 Latency improves after restart [ ] [ ] [ ]

Situation: Restarting the service immediately restores latency, but the problem returns a few hours later.

"What does that tell you, and what would you inspect next?"

Answer: A restart resets process-local state, so I would suspect memory pressure, fragmented pools, leaked connections, stuck threads, unbounded queues, cache growth, or a background task accumulating work. I would capture thread dumps, heap data, pool metrics, and queue depth before restarting whenever customer impact allows. I would correlate degradation with uptime rather than wall-clock time and compare a fresh instance with an old one under the same traffic. Restarting is a containment action, not a fix. The permanent change should bound or release the growing resource and alert on the leading indicator before latency fails.

2. Capacity, Scaling, and Traffic Management

S011 Sudden ten-times traffic increase [ ] [ ] [ ]

Situation: A celebrity shares a post and traffic jumps to ten times normal within two minutes.

"How would you keep the system available?"

Answer: I would protect the critical read path first. Autoscaling can add instances, but it may arrive too slowly and can overload databases, so I would use cached or precomputed content, CDN delivery, request coalescing, rate limits, and load shedding for nonessential features. I would monitor database connections, cache hot keys, queue depth, and dependency quotas while scaling gradually. Writes such as analytics or notifications can be buffered asynchronously. After the event, I would measure the actual bottleneck, increase tested burst capacity, pre-warm likely content, and document which features degrade safely under exceptional demand.

S012 Autoscaling adds servers but latency stays high [ ] [ ] [ ]

Situation: The autoscaler doubles application instances, but p95 latency barely changes.

"Why might horizontal scaling fail to help?"

Answer: More application instances help only when the application tier is the bottleneck and the workload is parallelizable. I would check whether all instances wait on the same database, cache, queue partition, external API, lock, or small connection pool. New instances may even worsen a shared bottleneck by opening more connections or producing more retries. I would use traces and saturation metrics to identify the constrained resource, then cap concurrency and protect it. The long-term fix may be query optimization, partitioning, caching, increasing dependency capacity, or redesigning serialized work rather than continuing to scale stateless servers.

S013 Scale-up or scale-out decision [ ] [ ] [ ]

Situation: A service is approaching CPU and memory limits, and the team debates using larger machines or more machines.

"How would you choose between vertical and horizontal scaling?"

Answer: I would consider workload parallelism, state, failure tolerance, scaling speed, operational complexity, and cost. A larger machine is simple and may help a stateful database or memory-heavy workload, but it has a ceiling and increases the impact of one failure. More machines improve redundancy and elastic capacity when requests can be distributed, but require load balancing, statelessness, coordination, and partition-aware dependencies. I would benchmark both at realistic load and choose the simplest option that meets the growth horizon. Often the near-term answer is a safe vertical increase while preparing horizontal scaling before the next limit.

S014 Uneven traffic across instances [ ] [ ] [ ]

Situation: Two service instances are overloaded while six others are mostly idle.

"What could cause this imbalance?"

Answer: I would inspect load-balancer policy, sticky sessions, long-lived connections, client-side discovery, health checks, and whether traffic is partitioned by a skewed key. Round-robin does not balance work if request cost varies greatly or connections remain pinned. I would compare request count and request cost per instance and check whether unhealthy instances are still advertised. A short-term response may be draining hot instances or disabling unnecessary affinity. Long term, I would use least-loaded or latency-aware balancing where appropriate, distribute hot keys, and ensure autoscaling reacts to per-instance saturation rather than only cluster averages.

S015 One tenant consumes most capacity [ ] [ ] [ ]

Situation: A single enterprise customer generates 60 percent of traffic and degrades service for everyone else.

"How would you protect fairness without unexpectedly breaking that customer?"

Answer: I would measure usage by tenant and introduce explicit quotas, concurrency limits, and weighted fairness rather than a single global limit. The immediate containment could cap the tenant's most expensive operation while preserving essential requests, accompanied by communication if contracts are affected. I would separate high-volume batch work from interactive traffic and consider dedicated capacity for negotiated tiers. Limits should return clear retry guidance and be monitored for false positives. The long-term design aligns capacity and pricing with tenant demand so one workload cannot exhaust shared threads, connections, or database resources.

S016 Read-heavy service is database-bound [ ] [ ] [ ]

Situation: The database CPU is saturated because a product page receives millions of repeated reads.

"What would you change first?"

Answer: I would verify that the repeated reads return data that can tolerate some staleness, then add caching at the most effective layer: CDN for public responses, application or distributed cache for shared objects, and request coalescing for simultaneous misses. I would define keys, TTL, invalidation, and behavior when the cache fails before rollout. Index tuning and read replicas may also help, but they still serve every request. I would watch cache hit rate, database load, and stale-data complaints. The design should prevent a cold-cache stampede so the cache does not merely postpone the same database overload.

S017 Write throughput has reached one database limit [ ] [ ] [ ]

Situation: A single primary database cannot sustain growing write traffic.

"What options would you evaluate before sharding?"

Answer: I would first reduce unnecessary writes, batch or debounce updates, optimize indexes, shorten transactions, and move noncritical side effects to asynchronous processing. I would confirm whether storage, CPU, locks, or connection count is the true limit. Vertical scaling and a stronger database configuration may buy time. Sharding is justified when one primary cannot meet the durable write requirement, but it introduces routing, rebalancing, cross-shard query, and transaction complexity. I would choose a stable, high-cardinality shard key, plan migrations and hot-key handling, and keep an escape path for repartitioning.

S018 Scale-down causes instability [ ] [ ] [ ]

Situation: After traffic falls, autoscaling removes instances and latency immediately oscillates up and down.

"How would you stabilize the autoscaler?"

Answer: The policy may react too quickly or to a noisy metric. I would inspect cooldown periods, stabilization windows, minimum replicas, startup and warm-up time, and whether average CPU hides overloaded instances. Scale-down should be slower than scale-up and should drain connections gracefully. I would use a demand signal that leads saturation, such as concurrency or queue depth, and require sustained low utilization before removal. I would also keep enough spare capacity for normal bursts. The goal is not minimum instance count every minute; it is predictable service with acceptable cost and without repeated cold starts.

S019 Capacity plan for a launch [ ] [ ] [ ]

Situation: Product expects a new feature to attract five million users in its first week, but the forecast is uncertain.

"How would you create a capacity plan?"

Answer: I would turn the product forecast into read and write requests per second, peak-to-average ratio, payload size, storage growth, fan-out work, and dependency calls. I would document assumptions and produce low, expected, and high scenarios instead of one precise number. Load tests should identify service and dependency limits with safety margin, while dashboards and feature flags support controlled rollout. I would pre-provision slow-scaling resources, confirm quotas, and define degradation and rollback thresholds. After launch, I would replace estimates with measured demand and revise the model rather than treating the initial spreadsheet as truth.

S020 Backpressure during overload [ ] [ ] [ ]

Situation: A service accepts work faster than it can process it, memory grows, and eventually the process crashes.

"How would backpressure change the behavior?"

Answer: Backpressure makes overload visible to callers instead of allowing an unbounded queue to consume memory. I would bound the queue and concurrency, reject or delay excess work with a clear retry signal, and prioritize critical requests. If asynchronous processing is suitable, a durable broker can absorb a limited burst, but queue depth and oldest-message age still need hard thresholds. Callers should use capped retries with jitter so rejection does not become a retry storm. Capacity must ultimately match sustained arrival rate; backpressure protects the system while traffic is reduced or processing capacity is restored.

3. Databases, Transactions, and Data Modeling

S021 Slow query after data growth [ ] [ ] [ ]

Situation: A query that took 40 ms now takes 2 seconds after the table grew from one million to one hundred million rows.

"How would you investigate and fix it?"

Answer: I would capture the exact query and execution plan with production-like parameters, because data distribution can change the chosen plan. I would inspect scanned rows, index usage, sorts, joins, filters, and whether statistics are current. The right fix may be a composite or covering index, a rewritten query, pagination, partition pruning, or precomputed data; adding every possible index would slow writes and consume storage. I would test the proposed plan on realistic volume and watch write cost after deployment. A query-latency alert by shape can reveal the next regression before users do.

S022 Duplicate records after retries [ ] [ ] [ ]

Situation: Clients retry a create request after timeouts, and the database contains duplicate orders.

"How would you make the operation safe to retry?"

Answer: I would make the create operation idempotent by requiring or generating an idempotency key that represents the client's logical request. The server stores the key with the result under a uniqueness constraint and returns the original result when the same key is retried. The check and write must be atomic; an in-memory map is not enough across instances. I would define key scope and expiry and reject reuse with different payloads. Timeouts can still leave the client uncertain, but a status lookup by key lets it learn the committed result without creating another order.

S023 Deadlocks in production [ ] [ ] [ ]

Situation: Database logs show deadlocks during peak traffic, and some requests fail even though CPU is normal.

"What causes this, and how would you reduce it?"

Answer: A deadlock occurs when transactions hold locks the others need and none can proceed. I would inspect the database deadlock graph to identify statements, tables, and lock order, then correlate them with application transactions. Immediate protection is to retry the aborted transaction a small number of times with jitter, but retries do not remove the cause. I would make code acquire resources in a consistent order, shorten transactions, reduce rows locked, add useful indexes, and move external calls outside the transaction. I would test concurrent execution because single-request tests rarely expose the cycle.

S024 Read replica returns stale data [ ] [ ] [ ]

Situation: A user updates their profile and immediately reads the old value because the read went to a replica.

"How would you provide a better user experience?"

Answer: Replication is asynchronous, so the replica can lag behind the primary. For read-after-write behavior, I could route that user's reads to the primary for a short period, attach a version and wait until a replica has applied it, or update the client optimistically when safe. I would not send every read to the primary unless the consistency requirement justifies the capacity cost. The chosen policy should be explicit per operation: account changes may need fresh reads, while feed browsing can tolerate staleness. Replication-lag metrics and fallback behavior are necessary when lag exceeds the expected window.

S025 Hot database shard [ ] [ ] [ ]

Situation: Data is sharded by user ID, but one celebrity account produces far more reads and writes than any other shard key.

"How would you handle this hot key?"

Answer: More shards do not help if all work for one key still lands on one shard. I would separate the read and write problems: aggressively cache public reads, serve media through a CDN, batch counters, and move fan-out work to queues. If the data model permits, I could split a hot logical key into subpartitions using another dimension such as post or time, while maintaining a directory for routing. That adds read aggregation and migration complexity, so I would reserve it for sustained hotspots. Rate limits and priority controls protect the primary while the data is redistributed.

S026 Cross-shard transaction requirement [ ] [ ] [ ]

Situation: A transfer must debit an account on one shard and credit an account on another shard.

"How would you preserve correctness?"

Answer: I would first ask whether related accounts can be colocated or whether the workflow can use a ledger on one authoritative partition. If not, a distributed transaction can provide atomicity but increases coordination, latency, and failure modes. A saga is often more practical: record a durable transfer intent, perform each step idempotently, and compensate or reconcile if a later step fails. The user-visible state must distinguish pending from complete rather than pretending the transfer is instantaneous. An immutable ledger, uniqueness keys, and a reconciliation job provide evidence and repair when messages or workers fail.

S027 Schema change on a large table [ ] [ ] [ ]

Situation: You need to add a required column to a table with hundreds of millions of rows without downtime.

"How would you roll out the migration safely?"

Answer: I would use an expand-and-contract sequence. First add a nullable column or a compatible default without requiring old code to write it. Deploy code that can read both forms and writes the new field, then backfill historical rows in small throttled batches while monitoring locks, replication lag, and database load. After validation, enforce the constraint and later remove old behavior. I would make the backfill restartable and pause it during pressure. A single blocking migration that rewrites the table may be operationally simple but is unsafe when its lock or duration is uncertain.

S028 Pagination slows on later pages [ ] [ ] [ ]

Situation: An API uses `OFFSET` pagination, and page 5,000 is much slower than page 1.

"What would you change?"

Answer: Large offsets force the database to find and discard many earlier rows, so cost grows with the page number and concurrent inserts can shift results. I would use keyset pagination with a stable ordering, such as `(created_at, id)`, and return an opaque cursor containing the last seen values. The next query asks for rows after that cursor using a matching index. This is fast and stable for forward navigation but does not naturally support arbitrary page numbers. I would document that trade-off and include a unique tie-breaker so equal timestamps do not duplicate or skip rows.

S029 Cache and database disagree after a write [ ] [ ] [ ]

Situation: A profile update succeeds in the database, but some users continue seeing the old cached value.

"How would you design the write and invalidation flow?"

Answer: The database should remain the source of truth, and the cache policy must define the failure window. A common approach is cache-aside: commit the database update, then invalidate the cache key so the next read reloads fresh data. Deleting is safer than writing the cache before the database because a failed transaction could publish nonexistent state. There is still a race with concurrent reads, so versioned values, short TTLs, or an event-driven invalidation can reduce risk. I would measure stale-read tolerance and include repair behavior when invalidation delivery fails.

S030 Data model for likes at large scale [ ] [ ] [ ]

Situation: A post may receive millions of likes, and the product needs both the count and whether the current viewer liked it.

"How would you model and serve this efficiently?"

Answer: I would store the user-post relationship with a uniqueness constraint so one user can like a post once, partitioned to support the dominant lookup. The exact count can be computed from relationships at small scale, but at high scale I would maintain a counter asynchronously or atomically with clear reconciliation because hot posts create write contention. Viewer state should be fetched in batches for all posts on a page, not one query per post. The displayed count may be slightly stale if product permits, while the user's own like action should reflect immediately and remain idempotent.

4. Caching and Content Delivery

S031 Cache stampede after expiry [ ] [ ] [ ]

Situation: A popular cache key expires and thousands of requests hit the database at once.

"How would you prevent this stampede?"

Answer: I would avoid letting every miss independently rebuild the same value. Options include request coalescing with one loader, a short distributed lock with timeout, stale-while-revalidate, proactive refresh, and randomized TTLs so many keys do not expire together. The fallback must be bounded; waiting forever for a failed loader simply moves the outage. I would monitor miss rate, loader latency, and database impact, and test cold-start behavior. For extremely hot content, serving a slightly stale value is often safer than forcing the origin to recompute it during a burst.

S032 Cache hit rate suddenly drops [ ] [ ] [ ]

Situation: Cache hit rate falls from 92 percent to 35 percent after a release, while traffic is unchanged.

"What would you investigate first?"

Answer: I would compare key construction, namespace or version changes, TTLs, eviction rate, memory pressure, serialization failures, and whether the release changed request cardinality. A harmless-looking new key component can turn one shared entry into millions of unique entries. I would split hit rate by endpoint and instance and check whether clients are reaching the same cache cluster. If origin load is dangerous, I would roll back or temporarily restore the old key strategy. Afterward, I would add cardinality and eviction alerts so a syntactically valid but ineffective cache is detected quickly.

S033 Cache contains sensitive data [ ] [ ] [ ]

Situation: A shared cache accidentally serves one user's private profile data to another user.

"What failed, and how would you contain it?"

Answer: The cache key or response-cache policy failed to include the authorization context. I would disable or purge the affected cache immediately, stop serving the endpoint if privacy exposure continues, and start an incident process to determine scope. The fix is to avoid shared caching for user-specific responses or include a stable user and permission dimension without putting secrets in the key. Authorization must still run on every request; a cache hit is not proof of access. I would add cross-user isolation tests, safe cache headers, audit evidence, and a review of any other personalized endpoints using the same layer.

S034 Redis becomes unavailable [ ] [ ] [ ]

Situation: The distributed cache is down, and all requests fall back to the primary database.

"How would you avoid turning a cache outage into a database outage?"

Answer: I would treat the cache as a load-bearing dependency even if data can theoretically be recomputed. The application should fail fast on cache calls, cap fallback concurrency, use local or stale values for safe data, and shed optional traffic before the database is saturated. Circuit breakers prevent every request from waiting on the failed cache, while gradual cache recovery avoids a synchronized refill. I would monitor database headroom and temporarily restrict expensive endpoints. Long term, I would test cache-loss scenarios and size the protected origin path for a deliberate, limited fallback rather than unlimited pass-through.

S035 CDN serves old content after update [ ] [ ] [ ]

Situation: A user replaces an image, but the CDN continues serving the old bytes from the same URL.

"How would you fix the design?"

Answer: Reusing a mutable URL conflicts with long-lived edge caching. I would store media under an immutable, content- or version-based URL and update the database reference when a new image is ready. Old URLs can remain cached safely and expire later. If immediate correction is required, a CDN purge can contain the issue, but purges are slower, costly, and harder to reason about at scale. I would separate metadata from media bytes so profile data can change quickly while immutable objects receive long cache lifetimes. Clients should not guess versions; the API returns the current canonical URL.

S036 Choosing a cache TTL [ ] [ ] [ ]

Situation: The team asks for one standard TTL for every cached object.

"How would you choose TTLs instead?"

Answer: TTL should follow each object's freshness requirement, update frequency, recomputation cost, popularity, and impact of stale data. Public profile images can live much longer than account permissions or inventory. I would define a bounded staleness target with product and security stakeholders, then use invalidation when prompt changes matter and TTL as a safety net. I would add jitter to reduce synchronized expiry and measure both hit rate and stale-read incidents. One global TTL is easy to configure but either wastes cache value on stable data or keeps volatile data stale for too long.

S037 Hot key overloads one cache node [ ] [ ] [ ]

Situation: A single viral post key receives most requests and saturates one cache node.

"How would you distribute the load?"

Answer: Standard key partitioning cannot spread one key because all requests hash to the same owner. I would first serve eligible content through a CDN or application-local cache so requests do not reach the distributed cache. Request coalescing and stale values reduce refill pressure. For very hot small objects, controlled key replication or sharded replicas can spread reads, but updates and invalidation become more complex. I would detect hot keys directly rather than relying only on average node load. The design should also protect the database if the hot entry disappears or the cache node fails.

S038 Cache warming during deployment [ ] [ ] [ ]

Situation: Every new instance starts with an empty local cache, causing a latency spike during deployments.

"How would you make startup safer?"

Answer: I would keep new instances out of full traffic until readiness checks pass and a limited warm-up completes. Warm only the small set of predictable hot data; loading the entire cache delays startup and may overload dependencies. A shared cache or stale snapshot can reduce per-instance misses, and a slow traffic ramp lets real requests populate entries gradually. I would randomize rollout timing and monitor origin load. The instance should still work correctly on a miss, because prewarming is an optimization rather than a correctness requirement, and deployment rollback must not depend on completing it.

S039 Negative caching for missing data [ ] [ ] [ ]

Situation: Bots repeatedly request nonexistent usernames, causing expensive database misses.

"Would you cache not-found results?"

Answer: Yes, if I use a short negative-cache TTL and understand the creation race. Caching a not-found result prevents repeated database lookups for the same invalid key, but a long TTL could hide a username created moments later. I would invalidate the negative entry when creation succeeds or keep the TTL short enough for the product requirement. I would distinguish a real not-found response from a database error so failures are not cached as absence. Rate limiting remains necessary because attackers can generate unlimited unique names and bypass per-key caching.

S040 Local cache versus distributed cache [ ] [ ] [ ]

Situation: The team wants the lowest possible latency and proposes replacing Redis with per-instance memory caches.

"What trade-offs would you explain?"

Answer: Local caches are extremely fast and remove a network hop, but each instance holds different data, duplicates memory, and needs a strategy for invalidation and cold starts. A distributed cache provides shared state and better aggregate memory use but adds network latency and another service dependency. I would choose based on consistency and access patterns, and often combine them: a very small short-lived local cache in front of a distributed cache. I would prevent unbounded memory growth and measure stale behavior. If correctness depends on globally current values, neither cache should replace the authoritative datastore.

5. Messaging, Queues, and Asynchronous Work

S041 Queue backlog grows continuously [ ] [ ] [ ]

Situation: Producers publish 5,000 events per second, consumers process 3,000, and queue lag keeps growing.

"What would you do immediately and long term?"

Answer: The system is unstable because sustained arrival rate exceeds service rate. I would measure oldest-message age, partition lag, consumer errors, processing time, and downstream saturation before simply adding consumers. Immediate actions may include scaling consumers, pausing noncritical producers, prioritizing urgent events, and shedding optional work. If one partition or dependency is the limit, more consumers will not help. Long term, I would optimize or batch processing, choose enough partitions, enforce producer quotas, and define maximum acceptable lag. A queue absorbs bursts; it cannot solve a permanent throughput deficit.

S042 Duplicate message delivery [ ] [ ] [ ]

Situation: A consumer occasionally processes the same payment event twice after a restart.

"How would you make the consumer safe?"

Answer: Most practical brokers provide at-least-once delivery, so duplicates are expected when acknowledgement and processing are not one atomic action. I would make the business operation idempotent using a stable event or command ID and a durable uniqueness record in the same transaction as the state change. The consumer should acknowledge only after the transaction commits. For external side effects, I would use an outbox or provider idempotency key. I would test redelivery deliberately. Broker settings may reduce duplicates, but correctness should not depend on a promise that a message will arrive exactly once.

S043 Poison message blocks a partition [ ] [ ] [ ]

Situation: One malformed event fails repeatedly and prevents later events in the same partition from being processed.

"How would you recover without silently losing data?"

Answer: I would limit retries with exponential backoff, capture the failure reason, and move the event to a dead-letter queue after the threshold so the partition can continue. The dead-letter record must preserve payload, headers, offset, schema version, and error context, with alerts and an owner. I would fix the consumer or data, then replay through a controlled idempotent path rather than editing offsets blindly. For ordering-sensitive workflows, skipping may violate correctness, so I might quarantine that key instead of the entire partition. The recovery policy should be designed before the first poison message.

S044 Event published but database write fails [ ] [ ] [ ]

Situation: A service publishes `OrderCreated`, then its database transaction fails, leaving consumers with an event for an order that does not exist.

"How would you prevent this inconsistency?"

Answer: I would avoid a dual write that requires the database and broker to succeed independently. With a transactional outbox, the service writes the order and an outbox row in one database transaction. A separate publisher reads committed outbox rows and sends them to the broker, retrying safely with event IDs. Consumers remain idempotent because publication can repeat. The reverse problem is handled too: an order cannot commit without an outbox record. This adds a relay, cleanup, and monitoring requirement, but it provides a durable bridge without a distributed transaction between unrelated systems.

S045 Consumer is fast but downstream is failing [ ] [ ] [ ]

Situation: Queue consumers rapidly retry calls to a failing email provider, increasing provider errors and queue lag.

"How would you stop the retry storm?"

Answer: I would add timeouts, capped retries with exponential backoff and jitter, and a circuit breaker that stops calls while the provider is unhealthy. Failed messages can be delayed or moved to a retry topic so they do not block fresh work or occupy workers. I would respect provider rate limits and separate permanent errors, such as invalid addresses, from temporary failures. During the outage, the queue provides durability, but I would watch retention and oldest-message age. Recovery should ramp gradually so accumulated messages do not immediately overload the provider again.

S046 Message ordering requirement [ ] [ ] [ ]

Situation: Profile update events sometimes arrive out of order, and an older event overwrites newer data.

"How would you preserve the correct final state?"

Answer: I would partition events by the entity key so updates for one profile are normally ordered, but I would still include a monotonically increasing version or source timestamp because retries and replays can reorder delivery. The consumer applies an event only if its version is newer than the stored version. Global ordering is unnecessary and expensive when only per-profile order matters. I would define how concurrent writers obtain versions and how deletes are represented. Monitoring rejected stale events helps detect upstream problems without allowing old state to overwrite current data.

S047 Exactly-once request from product [ ] [ ] [ ]

Situation: Product says a notification must be delivered "exactly once."

"How would you clarify and design for that requirement?"

Answer: I would ask whether the requirement means no duplicate user-visible notification, no duplicate provider call, or no duplicate internal event. End-to-end exactly-once delivery is rarely guaranteed across networks and external providers because acknowledgements can be lost. I would design at-least-once processing with stable notification IDs, idempotent state changes, deduplication at send time, and a durable delivery record. Some rare ambiguity may remain at the external boundary. I would explain that we can make duplicates extremely unlikely and observable, but claiming absolute exactly-once behavior without a shared transaction would be misleading.

S048 Replaying historical events [ ] [ ] [ ]

Situation: A new search indexer must rebuild its state from six months of events without affecting live traffic.

"How would you run the replay safely?"

Answer: I would use a separate consumer group and isolated capacity, starting from the required offset or archived event store. The indexer must be idempotent and schema-aware because old events may use earlier formats. I would throttle replay based on broker, database, and search-cluster headroom, and keep live updates flowing so the rebuilt index can catch up. Counts, checksums, and sampled comparisons validate completeness before traffic switches. If the source log does not retain six months, I would seed from a consistent database snapshot and replay events after the snapshot boundary.

S049 Choosing a queue partition key [ ] [ ] [ ]

Situation: You need to partition a high-volume event topic for user activity.

"How would you choose the partition key?"

Answer: The key should preserve only the ordering the business needs while distributing load evenly. User ID is suitable when events for one user must stay ordered and user activity is reasonably balanced. A constant key preserves global order but destroys parallelism; a random key scales well but loses per-user order. I would estimate key cardinality and skew, especially celebrity or bot accounts, and choose enough partitions for future throughput because repartitioning is operational work. Consumers and downstream stores must use the same ownership assumptions so state is not concurrently updated without coordination.

S050 Synchronous endpoint does too much work [ ] [ ] [ ]

Situation: Creating a post waits for image processing, search indexing, analytics, and notifications before returning.

"How would you redesign the request flow?"

Answer: I would keep only the work required to durably accept the post in the synchronous path: validate authorization, store metadata, and record reliable events or jobs. The API can return an accepted or processing state while workers generate media variants, update search, record analytics, and send notifications. Each worker must be idempotent, observable, and able to retry independently. The UI should show progress and a clear failure state instead of pretending all side effects completed. This lowers user latency and isolates failures, but it introduces eventual consistency and requires durable workflow tracking.

6. Reliability, Resilience, and Disaster Recovery

S051 Downstream dependency is slow [ ] [ ] [ ]

Situation: A recommendation service becomes slow and starts consuming all request threads in the feed API.

"How would you prevent the dependency from taking down the caller?"

Answer: I would set a timeout based on the caller's latency budget, limit concurrent calls with a bulkhead, and open a circuit breaker when failures or latency cross a threshold. The feed should have a defined fallback, such as cached recommendations, chronological content, or omitting the section. Retries must be few, budget-aware, and jittered because retrying a saturated dependency amplifies load. I would monitor fallback rate as a degraded-service signal. The long-term fix may be dependency scaling or a looser asynchronous design, but the caller must remain protected whenever the dependency fails again.

S052 Retry storm after a brief outage [ ] [ ] [ ]

Situation: A database is unavailable for 30 seconds, then receives a huge burst of retries and remains overloaded after recovery.

"What retry policy would avoid this?"

Answer: I would use exponential backoff with random jitter, a strict maximum attempt count, and a total request deadline. Retries should occur only for operations known to be transient and safe to repeat; writes need idempotency. A circuit breaker can stop futile calls during the outage, while concurrency limits protect the recovering database. Clients should not all retry at the same fixed interval. I would also review timeouts: a long timeout plus several retries can exceed user patience and hold resources. Recovery should reopen gradually so the database proves it has headroom before receiving full traffic.

S053 One dependency fails but core service should remain usable [ ] [ ] [ ]

Situation: The analytics service is down, but users should still be able to upload photos.

"How would you degrade gracefully?"

Answer: Analytics should not be on the critical upload path. I would durably accept the upload and publish or store an analytics event asynchronously. If the broker is unavailable, an outbox in the upload database can preserve the event without blocking the user indefinitely. The application should apply a short timeout and skip nonessential synchronous analytics calls. I would expose a degraded-state metric and alert because silent loss is not acceptable. Once analytics recovers, it can replay idempotently. The design explicitly separates product correctness from secondary measurement so an optional dependency cannot become mandatory by accident.

S054 Multi-region failover decision [ ] [ ] [ ]

Situation: The primary region is experiencing widespread network failures, and traffic may need to move to a secondary region.

"What would you verify before failing over?"

Answer: I would confirm the failure scope, secondary-region health and capacity, data replication lag, DNS or routing controls, and whether both regions could accept conflicting writes. If the system is active-passive, I would fence the old primary or make it read-only before promoting the secondary to avoid split brain. I would compare expected data loss with the recovery point objective and expected downtime with the recovery time objective. Traffic should shift gradually where possible. After recovery, I would reconcile data and rehearse failback rather than immediately reversing direction under pressure.

S055 Backup exists but restore was never tested [ ] [ ] [ ]

Situation: The team takes daily backups and assumes disaster recovery is covered.

"Why is that insufficient?"

Answer: A backup is useful only if it is complete, protected, and restorable within the required time. I would schedule automated restore tests into an isolated environment, verify checksums and schema compatibility, and run application-level checks on restored data. I would measure actual restore duration and data age against recovery objectives. Credentials, encryption keys, configuration, and dependent object storage may also be required; a database file alone may not recreate the service. I would retain multiple versions in a separate failure domain and alert on missed backups. Recovery evidence matters more than a successful backup job status.

S056 Partial network partition [ ] [ ] [ ]

Situation: Half the service instances can reach the database and half cannot, while all still pass basic process health checks.

"How should health checks and routing behave?"

Answer: A liveness check should confirm the process can make progress without causing constant restarts, while readiness should remove an instance from traffic when it cannot serve required requests. I would make readiness reflect critical dependency reachability with careful timeouts and hysteresis so brief failures do not flap every instance at once. If some endpoints can work without the database, separate routing or degraded behavior may preserve them. I would compare network paths and zones to isolate the partition. The system should avoid reporting success while requests fail, but health checks must not create a synchronized self-evacuation during dependency pressure.

S057 Leader election split brain [ ] [ ] [ ]

Situation: Two scheduler instances both believe they are leader and execute the same cleanup job.

"How would you prevent duplicate leadership from corrupting data?"

Answer: Leader election alone is not enough because an old leader can continue working after losing its lease during a pause or partition. I would use a lease with expiry and a monotonically increasing fencing token. Every protected write includes the token, and the storage layer rejects work from an older leader. Jobs should also be idempotent when possible. I would inspect clock assumptions and use a proven coordination service rather than custom in-memory voting. Monitoring should alert on overlapping leadership and duplicate job IDs so the protection is tested before a destructive task exposes the flaw.

S058 Cascading failure across services [ ] [ ] [ ]

Situation: One slow service causes callers to retry, their thread pools fill, and unrelated endpoints begin failing.

"How would you stop the cascade?"

Answer: I would contain the initial dependency with short bounded timeouts, circuit breakers, bulkheads, and concurrency limits so its callers retain resources for other work. Retries should respect a shared deadline and retry budget rather than multiplying at every layer. Load shedding and priority queues keep critical operations available. I would trace the dependency graph to identify where latency and retries amplify, then disable optional calls or use cached fallbacks. Long term, I would run failure tests that slow one dependency and verify the blast radius, because ordinary happy-path load tests do not expose cascading behavior.

S059 Disaster destroys primary data store [ ] [ ] [ ]

Situation: The primary database and its local replicas are permanently lost.

"Walk through recovery priorities."

Answer: I would declare the incident, stop unsafe writes, and confirm the latest restorable backup or cross-region replica and its data age. Recovery follows the documented runbook: provision a clean store, restore data, apply logs if available, validate integrity and application compatibility, then direct limited traffic while watching errors. I would communicate the expected recovery point and any confirmed data loss rather than guessing. After service restoration, reconciliation may recover operations from durable queues or external records. The post-incident work must address why replicas and backups shared a failure domain and repeat the restore drill.

S060 Graceful shutdown during deployment [ ] [ ] [ ]

Situation: Instances terminate during rolling deployment and active requests fail with connection resets.

"How would you make termination graceful?"

Answer: On termination, the instance should first fail readiness so the load balancer stops sending new traffic, then wait for routing changes to propagate and drain active requests within a bounded grace period. Background consumers should stop taking new messages, finish or safely abandon current work, and acknowledge only committed work. Long-lived connections may need explicit close signals or reconnection guidance. I would align application shutdown timeouts with platform termination settings and test slow requests. Graceful shutdown reduces expected errors, but hard deadlines remain necessary so a stuck process cannot block deployment indefinitely.

7. APIs, Consistency, and Correctness

S061 Client retries a timed-out write [ ] [ ] [ ]

Situation: A client times out while creating a post and does not know whether the server committed it.

"What should the API allow the client to do?"

Answer: The API should accept an idempotency key so the client can safely retry the same logical create. The server atomically stores the key, request identity, status, and result with the post. If the original request committed, the retry returns the same post; if it did not, one attempt creates it. A status endpoint by key can resolve longer-running uncertainty. I would reject the same key with a different payload and define retention. Without this contract, the client must choose between duplicate posts and potentially losing a valid user action.

S062 API version breaks old mobile clients [ ] [ ] [ ]

Situation: A backend response removes a field, and users on older mobile versions start crashing.

"How would you manage compatibility?"

Answer: Mobile clients can remain in the field for months, so server changes should be backward compatible by default. I would restore the field or roll back immediately, then use additive changes, tolerant readers, defaults, and explicit deprecation metrics. If a breaking contract is unavoidable, I would introduce a versioned endpoint or negotiated representation and maintain both until usage falls below an agreed threshold. Contract tests using older client expectations should run before deployment. I would avoid reusing a field with new meaning because syntactic compatibility can still create incorrect behavior.

S063 Concurrent updates overwrite each other [ ] [ ] [ ]

Situation: Two devices edit the same profile, and the later database write silently erases the earlier change.

"How would you prevent lost updates?"

Answer: I would use optimistic concurrency with a version number or ETag. The client reads version 12 and submits an update conditional on version 12; the database updates only if the current version still matches and increments it. If another write already produced version 13, the server returns a conflict so the client can reload, merge, or ask the user. This avoids long-held locks for normal low-contention edits. Field-level patch semantics may reduce conflicts, but the API must define whether absent fields mean unchanged or cleared. Silent last-write-wins is acceptable only when the product explicitly accepts data loss.

S064 Strong versus eventual consistency for follower count [ ] [ ] [ ]

Situation: A user follows another account, but the displayed follower count updates a few seconds later.

"Is eventual consistency acceptable here?"

Answer: I would separate relationship correctness from aggregate display. The follow edge should be committed durably and idempotently so authorization and future actions see the correct relationship. The public counter can often update asynchronously and be temporarily stale because exact immediate counts are not usually critical. The initiating user's UI can update optimistically while reconciliation corrects drift. I would document the expected delay and monitor lag. Strong consistency is worth its coordination cost for money, permissions, or uniqueness, but applying it to every social counter would reduce availability and throughput without proportional user value.

S065 Partial success in a batch API [ ] [ ] [ ]

Situation: A batch request contains 100 items; 93 succeed and 7 fail validation.

"What should the API return and retry behavior be?"

Answer: I would define atomicity explicitly. If items are independent, the response should report per-item status, stable identifiers, and actionable errors so the client retries only failed transient items. Successful items must not repeat, so idempotency keys or item IDs are required. If business rules require all-or-nothing behavior, the server should validate before commit or use a transaction within a supported boundary, accepting the capacity cost. A vague 500 response encourages retrying all 100 items and creating duplicates. Metrics should distinguish validation failures from infrastructure failures because their remedies differ.

S066 API returns too much data [ ] [ ] [ ]

Situation: A feed endpoint returns full comments, likes, profiles, and media metadata for every post, creating multi-megabyte responses.

"How would you redesign the contract?"

Answer: I would identify what the initial screen actually needs and return a bounded summary: post fields, a small comment preview, viewer state, and counts. Additional comments or details load through paginated endpoints. I would support field selection only if the complexity is justified and ensure authorization remains consistent. Compression helps bandwidth but does not remove database, serialization, memory, or client parsing cost. I would set response-size and item-count limits, measure payload percentiles, and version the contract compatibly. The goal is predictable work per request rather than an unbounded nested object graph.

S067 Webhook receiver sees duplicate callbacks [ ] [ ] [ ]

Situation: A payment provider sends the same webhook several times and sometimes out of order.

"How would you process it safely?"

Answer: I would verify the provider signature and timestamp before accepting the event, then store the provider event ID under a uniqueness constraint. Processing is idempotent, and state transitions validate version or allowed order so an older event cannot reverse newer state. I would acknowledge quickly after durable acceptance and perform slow work asynchronously, because a slow response triggers more retries. Unknown events should be retained for investigation rather than treated as success without evidence. A reconciliation job against the provider API closes gaps when callbacks are delayed or permanently missed.

S068 Public API rate limiting [ ] [ ] [ ]

Situation: A public endpoint is overwhelmed by one client sending thousands of requests per second.

"How would you design rate limiting?"

Answer: I would choose the identity and scope first: API key, user, IP, tenant, endpoint, or a combination. A token-bucket limit allows bounded bursts while controlling the sustained rate. Limits should be stricter for expensive writes than cheap reads and should return a clear status with retry guidance. A distributed limiter needs shared or partition-consistent state, while local limits are faster but approximate globally. I would include abuse detection, allowlists for trusted internal traffic, and metrics for rejected requests. The limiter protects capacity but does not replace authentication or endpoint optimization.

S069 Search endpoint supports arbitrary filters [ ] [ ] [ ]

Situation: Product wants users to combine dozens of filters and sorts on a rapidly growing dataset.

"Would you keep querying the primary relational database?"

Answer: I would start with actual query patterns, scale, freshness, and correctness needs. A relational database with targeted indexes may be simplest for modest data and transactional filters, but arbitrary full-text search and many combinations can create expensive plans and index explosion. A search engine offers inverted indexes and ranking but introduces asynchronous indexing, operational cost, and temporary inconsistency. I would keep the database authoritative, publish changes reliably, and rebuild the search index from a snapshot plus events. The API should expose bounded supported filters rather than promising every possible combination.

S070 Delete must propagate across systems [ ] [ ] [ ]

Situation: A user deletes a post that exists in the database, cache, search index, CDN, analytics pipeline, and backups.

"How would you design deletion?"

Answer: I would make the database deletion or tombstone the authoritative state and immediately prevent the post from being returned or authorized. A durable deletion event drives idempotent removal from caches, search, derived stores, and media references, with retries and completion tracking. CDN content may require purge or short-lived access controls; backups follow a documented retention and erasure policy rather than instant mutation. I would distinguish user-visible deletion latency from eventual physical cleanup and provide audit evidence. A reconciliation job finds missed consumers so one failed event does not leave content discoverable indefinitely.

8. Deployments, Observability, and Operations

S071 Canary looks healthy but full rollout fails [ ] [ ] [ ]

Situation: A 5 percent canary shows no problems, but latency rises after the release reaches 70 percent.

"Why did the canary miss it, and how would you improve the rollout?"

Answer: The regression may depend on aggregate load, cache churn, connection count, a rare traffic segment, or interaction between new instances. I would pause or roll back, then compare version-specific metrics and dependency pressure at each rollout step. Future canaries should receive representative traffic, run long enough to cover background behavior, and include guardrails for downstream metrics, not only canary CPU and errors. I would use progressive stages such as 1, 5, 20, 50, and 100 percent with automatic analysis and soak time. Some capacity failures require load testing in addition to canaries.

S072 Metrics are green but users report failures [ ] [ ] [ ]

Situation: Standard dashboards show normal CPU, average latency, and error rate, yet customer support reports many failed uploads.

"What might observability be missing?"

Answer: Aggregate metrics may hide one endpoint, tenant, region, file size, or client version. I would reproduce a failed request and segment success rate and latency by those dimensions, then inspect traces and structured logs using a request ID. The service may also return HTTP 200 while the asynchronous upload later fails, so business outcome metrics matter. I would add an end-to-end synthetic upload and track accepted, processed, and visible states. Observability should represent the user's journey, not only machine health, and should avoid unbounded labels such as raw user IDs.

S073 Logs are too noisy during incidents [ ] [ ] [ ]

Situation: An incident produces millions of repetitive error logs, making useful evidence difficult to find and increasing logging cost.

"How would you improve logging?"

Answer: I would use structured logs with stable fields such as service, operation, request ID, error class, dependency, and version. Expected transient failures should be sampled or aggregated, while security and data-loss events remain complete. I would avoid logging the same exception at every layer and ensure sensitive values are redacted. Metrics should count high-volume events; logs provide detailed examples. During the incident, I might temporarily reduce a noisy category without disabling audit evidence. Retention and indexing should reflect diagnostic value, and alerts should point to a focused query rather than a raw log stream.

S074 Missing distributed trace spans [ ] [ ] [ ]

Situation: A request crosses five services, but traces often stop after the second service.

"How would you diagnose the broken trace?"

Answer: I would check whether trace context is propagated through HTTP headers, messaging metadata, thread or reactive boundaries, and retry clients. Sampling decisions should remain consistent across services; otherwise children may be dropped even when the root is kept. I would verify instrumentation versions, exporters, collector capacity, and clock synchronization. A correlation ID in logs provides a fallback but is not a substitute for parent-child timing. I would add an automated end-to-end trace test across synchronous and asynchronous paths and monitor span-drop rate so observability failures are detected before a production incident.

S075 Alert fires too often and gets ignored [ ] [ ] [ ]

Situation: A latency alert pages the team several times a week but rarely requires action.

"How would you make the alert useful?"

Answer: I would tie paging to user-impacting symptoms and a documented action. Instead of a fixed average-latency threshold, I might alert on sustained p95 or p99 latency combined with request volume or error-budget burn. Short spikes can create tickets or dashboards rather than waking someone. I would review historical pages, remove redundant alerts, and include service, scope, likely dependencies, and runbook links. After every page, the owner should classify whether it was actionable. An alert that nobody trusts is not harmless; it delays response to the incident that actually matters.

S076 Need to roll back a database-compatible release [ ] [ ] [ ]

Situation: Application code must roll back, but the deployment already started writing a new database field.

"How do you make rollback safe?"

Answer: The migration and code should have been designed for mixed versions. During expansion, old code must tolerate the new field and new code should continue writing any old representation needed by the previous version. If that contract holds, I can roll application instances back without reversing the schema. Destructive cleanup happens only after the rollback window closes and all readers have migrated. If compatibility is uncertain during an incident, I would stop the rollout and test the previous version against current data before broad rollback. Schema rollback is usually riskier than leaving an unused additive column temporarily.

S077 Feature flag causes inconsistent behavior [ ] [ ] [ ]

Situation: Two services evaluate the same feature flag differently, producing requests one service cannot understand.

"How would you manage cross-service feature rollout?"

Answer: A flag is not a substitute for a compatible protocol. I would make the receiving service understand both old and new requests before any caller sends the new form. Flag ownership, targeting, defaults, and expiry should be explicit, and evaluation should use a consistent subject key when user experience must be stable. During the incident, I would disable the producer-side behavior or route only to compatible versions. I would audit configuration propagation delay and fail-safe defaults. Once rollout completes, I would remove the flag and old path so temporary branching does not become permanent architecture.

S078 Configuration change causes outage [ ] [ ] [ ]

Situation: No code was deployed, but a connection-pool configuration change caused widespread timeouts.

"How should configuration changes be managed?"

Answer: Configuration can change behavior as significantly as code, so it needs version control, review, validation, staged rollout, observability, and rollback. I would restore the previous value first if impact is clear, then verify whether the new pool size exhausted the database or created local waiting. Sensitive and environment-specific values remain in a secure configuration system, but their schemas and safe ranges can still be tested. I would record who changed what and when, compare effective configuration by instance, and use canary application of high-risk settings rather than broadcasting them globally.

S079 Load test passes but production fails [ ] [ ] [ ]

Situation: A service passes a synthetic load test at expected requests per second but fails under real traffic.

"What might the test have missed?"

Answer: Request rate alone is not a representative workload. I would compare payload sizes, endpoint mix, key distribution, cache warmth, connection reuse, think time, authenticated behavior, database volume, and dependency latency. Production may contain hot keys, large media, rare slow queries, or synchronized bursts that the test did not model. I would use sanitized traffic statistics to build multiple workload profiles and run long enough to expose memory or pool leaks. The result should report saturation points and failure behavior, not only a pass at one target. Production can then validate assumptions through a controlled canary.

S080 Runbook is outdated during an incident [ ] [ ] [ ]

Situation: The on-call engineer follows a runbook, but commands and dashboards no longer match the system.

"How would you keep operational documentation trustworthy?"

Answer: I would stop using unsafe steps, stabilize the service with current expertise, and record exactly where the runbook failed. Runbooks need named owners, review dates, tested commands, expected observations, rollback steps, and links that are validated automatically where possible. Incident drills and actual incidents should update them immediately. I would prefer small task-focused procedures over a large document nobody can rehearse. Automation should encode repetitive safe actions, but the runbook must still explain intent and decision boundaries. A stale runbook should be marked unverified rather than presented as authoritative.

9. Security, Privacy, and Abuse

S081 Credential-stuffing attack [ ] [ ] [ ]

Situation: Login failures rise sharply from many IP addresses using known email and password pairs.

"How would you protect accounts without locking out legitimate users?"

Answer: I would detect attempts by account, IP, device, network range, and reputation rather than relying on one global threshold. Progressive delays, risk-based challenges, breached-password checks, and multi-factor authentication can slow attackers while preserving legitimate recovery. Per-account limits should not let an attacker permanently deny service to a victim. I would avoid revealing whether an email exists and alert users on suspicious successful logins. Immediate containment may include blocking high-confidence sources and tightening limits. Long term, I would monitor takeover outcomes, not only failed attempts, and protect reset flows equally.

S082 Authorization bug exposes another user's data [ ] [ ] [ ]

Situation: A user changes an object ID in the URL and can view another user's private resource.

"What should happen immediately and how do you prevent recurrence?"

Answer: I would disable or restrict the affected endpoint, preserve audit evidence, determine exposure scope, and follow the security incident process. The root failure is object-level authorization: authentication proved who the caller was but the service did not verify access to that specific resource. I would centralize authorization checks close to data access, use owner or policy constraints in the query, and return the same safe response for unauthorized objects where appropriate. Tests must cover cross-user and cross-tenant access, including cached paths. Logging should identify access decisions without recording sensitive payloads.

S083 Secret appears in logs [ ] [ ] [ ]

Situation: An access token is accidentally written to centralized application logs.

"How would you contain and remediate the incident?"

Answer: I would revoke or rotate the exposed credential immediately, restrict access to the affected logs, and identify how widely the value was replicated or exported. Deleting one local file is insufficient when logs have indexes, backups, and downstream sinks. I would search for similar patterns without spreading the secret further, notify security owners, and preserve an audit trail. The code fix should use structured redaction at a common logging boundary and prohibit sensitive headers and fields. Tests and scanners can catch known secret formats, but developers also need a data-classification rule for values that scanners cannot recognize.

S084 File upload contains malicious content [ ] [ ] [ ]

Situation: Users can upload images, and an attacker uploads a file with a misleading extension and embedded malicious payload.

"How would you secure the upload pipeline?"

Answer: I would not trust the filename, extension, or client-provided content type. The service should enforce size limits, inspect file signatures, decode and re-encode supported media, scan in an isolated environment, and store uploads outside executable application paths. Newly uploaded objects remain quarantined until processing succeeds. Delivery uses a separate media domain with safe content headers and no authentication cookies. I would generate random storage keys and strip unnecessary metadata. Timeouts and resource limits protect image processors from decompression bombs, while failed or suspicious files are retained only according to a controlled investigation policy.

S085 Internal service trusts any network caller [ ] [ ] [ ]

Situation: A service assumes requests are safe because they come from the private network.

"Why is that risky, and what would you add?"

Answer: Network location is a weak identity boundary; compromised workloads, routing mistakes, or exposed proxies can make unauthorized calls. I would authenticate service identities using short-lived credentials or mutual TLS, authorize specific operations by least privilege, and encrypt traffic where required. Network policies reduce reachability but do not replace application authorization. I would rotate identities automatically and audit denied and sensitive actions. During migration, I would inventory callers and introduce enforcement gradually to avoid breaking unknown dependencies. The goal is explicit trust between workloads, not a blanket assumption that everything inside the network is safe.

S086 Multi-tenant data leakage risk [ ] [ ] [ ]

Situation: A SaaS database stores many tenants in shared tables, and the team worries about queries missing the tenant filter.

"How would you reduce the chance and impact of leakage?"

Answer: I would make tenant identity part of every authenticated request context and enforce it in a shared data-access layer rather than relying on each developer to remember a filter. Composite keys and uniqueness constraints should include tenant ID, and database row-level security can add defense in depth where supported. Cache keys, queues, object storage paths, and logs also need tenant boundaries. Automated tests should attempt cross-tenant access for every resource type. For high-risk or regulated tenants, stronger isolation such as separate schemas or databases may be justified, with added operational cost.

S087 DDoS traffic reaches application servers [ ] [ ] [ ]

Situation: A volumetric attack overwhelms network and application capacity before normal rate limits can respond.

"Where should defenses be placed?"

Answer: Volumetric traffic should be absorbed as far from the application as possible using the CDN, managed DDoS protection, edge filtering, and load-balancer limits. Application rate limiting is still useful for authenticated abuse and expensive operations, but it cannot help if network links or proxies are already saturated. I would cache public content, challenge suspicious clients, protect origin addresses, and preserve a small path for operators. The incident plan should include provider contacts and traffic baselines. Afterward, I would review which layers exhausted first and test that emergency rules do not block all legitimate users.

S088 User requests data deletion [ ] [ ] [ ]

Situation: A user asks for all personal data to be deleted from the platform.

"How would you make the deletion complete and auditable?"

Answer: I would maintain a data inventory mapping the user's identifiers across authoritative databases, derived stores, search, caches, object storage, analytics, and third parties. A durable workflow records the request, verifies authorization, deletes or irreversibly anonymizes each system idempotently, and tracks completion with retries. Backups follow the documented retention policy and prevent restored data from becoming active again. Legal retention exceptions must be explicit and access-controlled. The user-facing status should not claim completion until required systems acknowledge it, and audits should prove the workflow without retaining the deleted personal data itself.

S089 Signed media URL is shared [ ] [ ] [ ]

Situation: A private photo URL is signed for one user but can be copied and used by anyone until it expires.

"How would you choose a safer access model?"

Answer: A signed URL is a bearer credential, so anyone holding it can use it during its lifetime. I would keep expiry short, scope it to the exact object and operation, use HTTPS, and avoid placing it in logs or analytics. If stronger user binding is required, requests can pass through an authenticated media proxy or use edge authorization that validates a session, accepting extra latency and cost. I would revoke access by changing the object version or signing key only when necessary because broad key rotation affects many URLs. The design depends on privacy sensitivity and acceptable sharing risk.

S090 Audit log can be modified by application admins [ ] [ ] [ ]

Situation: The same administrators who operate the service can alter records intended to prove sensitive actions.

"How would you make audit evidence more trustworthy?"

Answer: I would send security-relevant events to an append-oriented system with separate access control, limited deletion rights, retention enforcement, and monitoring for gaps. Events include actor identity, action, target, timestamp, request or trace ID, result, and relevant policy version without unnecessary secrets. Application administrators may read operational details but should not be able to rewrite the independent record. Cryptographic chaining or immutable storage can strengthen tamper evidence for high-risk systems. I would test clock consistency, delivery failure behavior, and reconciliation so the absence of an event is detected rather than silently accepted.

10. Feeds, Media, Search, Ranking, and Notifications

S091 Celebrity fan-out overload [ ] [ ] [ ]

Situation: A celebrity with 50 million followers publishes a post, and fan-out-on-write would enqueue 50 million feed updates.

"How would you design around this hotspot?"

Answer: I would use a hybrid feed strategy. Normal accounts can fan out on write for fast reads, while celebrity posts remain in an author stream and are merged into follower feeds at read time or through staged batch distribution. This avoids one write creating an immediate massive burst, though reads become more complex and may need caching. I would define the celebrity threshold dynamically from fan-out cost, monitor queue lag, and make duplicate insertion idempotent. The ranking layer should treat pulled and pushed candidates consistently so the optimization does not change product semantics unexpectedly.

S092 Feed shows duplicate posts [ ] [ ] [ ]

Situation: Users sometimes see the same post twice after refreshing or loading the next page.

"Where could duplicates enter, and how would you remove them?"

Answer: Duplicates can come from repeated fan-out events, overlapping pagination windows, merging several candidate sources, or retries after uncertain writes. I would assign a stable post ID and deduplicate at durable insertion where possible, then deduplicate again during feed assembly as defense in depth. Cursor pagination should use a stable score plus unique tie-breaker so page boundaries do not overlap. I would trace duplicate pairs to their source rather than hiding every error in the UI. Consumer idempotency and unique feed-entry constraints prevent the stored timeline from growing with duplicate work.

S093 Feed freshness versus ranking quality [ ] [ ] [ ]

Situation: A sophisticated ranking model improves engagement but takes several seconds to update after a new post.

"How would you balance freshness and ranking?"

Answer: I would define a latency budget and combine candidate paths. A fast path can inject recent eligible posts using simple rules, while the main ranked feed uses richer features and precomputation. The merger removes duplicates and enforces safety and visibility rules. I would measure freshness, engagement, latency, and compute cost rather than optimizing one metric alone. If the model or feature service is slow, a cached score or chronological fallback keeps the feed available. The product may accept slightly lower ranking precision for the first seconds after publication to avoid making new content invisible.

S094 Image processing backlog [ ] [ ] [ ]

Situation: Original images upload successfully, but thumbnail generation is hours behind.

"How would you restore service and prevent recurrence?"

Answer: I would inspect queue age, worker errors, processing time by image type, and CPU or memory saturation. Immediate actions include scaling healthy workers, quarantining poison files, prioritizing user-visible variants, and limiting new work if storage or queues approach limits. Upload responses should show a processing state and never reference a thumbnail that does not exist. Long term, I would make jobs idempotent, enforce input limits, generate common variants first, and autoscale on oldest-job age rather than only CPU. Reprocessing tools and source-image retention provide recovery when a processor bug creates bad outputs.

S095 Video upload interrupted halfway [ ] [ ] [ ]

Situation: A user uploads a large video over an unreliable mobile connection and loses connectivity at 70 percent.

"How would you support reliable resume?"

Answer: I would use multipart or chunked upload with an upload session ID. Each part has a sequence number and checksum and can be retried idempotently, ideally directly to object storage using short-lived signed permissions. The server tracks received parts and finalizes only when all expected parts are present and validated. Incomplete sessions expire and are cleaned up. The client asks which parts already exist rather than restarting from zero. Authorization is checked at session creation and finalization, and processing begins only after the immutable original object is complete.

S096 Search index misses recent posts [ ] [ ] [ ]

Situation: New posts appear in the database but are not searchable for several minutes.

"How would you diagnose and bound indexing delay?"

Answer: I would trace the path from database commit to outbox, broker, indexer, and search refresh, measuring lag at each boundary. A missing event, consumer backlog, poison document, or slow index refresh creates different remedies. The post remains authoritative in the database, and the product should communicate or tolerate the documented indexing delay. I would alert on oldest unindexed event, use idempotent upserts, and provide replay from a known offset. For critical read-after-write behavior, the UI can surface the newly created post directly without requiring immediate search consistency.

S097 Ranking feature service is unavailable [ ] [ ] [ ]

Situation: The online feature store used by the ranking model becomes unavailable during peak traffic.

"How should the feed degrade?"

Answer: Ranking should have a bounded feature-fetch timeout and a predefined fallback. I would use cached or default features, a simpler model with locally available signals, or chronological ranking rather than failing the entire feed. The response should still enforce privacy, blocking, and safety filters because those are correctness constraints, not optional ranking features. A circuit breaker prevents every request from waiting on the store, and fallback rate becomes an operational metric. After recovery, traffic ramps back gradually. Long term, I would replicate critical features or precompute them according to freshness requirements.

S098 Notification storm [ ] [ ] [ ]

Situation: A bug triggers millions of duplicate push notifications for the same event.

"How would you stop the storm and protect users?"

Answer: I would disable the producing feature or notification campaign immediately, pause affected consumers, and preserve event IDs for investigation. The sending path should enforce a deduplication key based on user, event, channel, and notification type, plus per-user and global rate limits. A kill switch must work independently of the faulty producer. I would distinguish queued but unsent messages from provider-accepted messages because some cannot be recalled. Recovery should discard confirmed duplicates and resume gradually. Tests should cover producer retries and consumer restarts so duplicate internal delivery never becomes duplicate user-visible delivery.

S099 Comments count and list disagree [ ] [ ] [ ]

Situation: A post displays 120 comments, but opening the list shows only 116 because some comments were deleted or moderation is delayed.

"How would you reason about the inconsistency?"

Answer: I would define what the count represents: all stored comments, visible comments for this viewer, or an approximate public aggregate. If deletions and moderation update the list and counter asynchronously, short disagreement may be acceptable, but the UI should not promise exactness. I would make counter updates idempotent, emit events for visibility changes, and run reconciliation against authoritative comment state. Viewer-specific filtering can mean different users legitimately see different counts, so a single global exact counter may be impossible. Product requirements determine whether to display an approximation, hide the count temporarily, or compute a costlier filtered value.

S100 Design an Instagram-like home feed under failure [ ] [ ] [ ]

Situation: You are asked to design a home feed that remains useful when ranking, notification, or analytics dependencies fail.

"What is your high-level design and graceful-degradation strategy?"

Answer: I would keep post creation durable in the primary store, publish changes through an outbox, and build feed candidates using hybrid fan-out: push for normal accounts and pull for celebrities. The read service merges candidates, applies visibility rules, ranks them, hydrates metadata in batches, and serves media through a CDN. Caches reduce repeated reads, while cursor pagination bounds work. Ranking has a chronological or cached fallback; analytics and notifications are asynchronous and cannot block reads. Timeouts, circuit breakers, idempotent consumers, lag metrics, and replay support contain failures. The critical promise is a safe usable feed, not that every enhancement is always fresh.

60-SECOND ANSWER SHAPE

1. State the leading hypothesis without claiming certainty. **2.** Name the first metrics, traces, or logs to compare. **3.** Contain customer impact before deep investigation. **4.** Isolate the bottleneck or failed boundary. **5.** Finish with the durable fix, trade-off, and prevention signal.

MORNING SCENARIO REVIEW LOG

Record the scenarios you attempted and the part of your answer that needs repair before the next recall.

Date	Drill IDs	Weak spot / repair	/5